

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: JHONY ALBERTO AGUILAR RODRIGUEZ

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad _____

Actividad # 1

Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Objetivo de la tarea:

- Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.



ANÁLISIS DEL PROBLEMA

- Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas).

Descripción general del problema

En Data Center, la gestión de incidentes técnicos es un proceso crítico que requiere organización, rapidez y trazabilidad. Actualmente, la falta de un sistema estructurado puede provocar retrasos en la atención de fallos, mala asignación de técnicos y pérdida de información relevante sobre los incidentes.

Se propone el diseño de un sistema de gestión de incidentes (Help Desk), que permita registrar, clasificar, asignar y dar seguimiento a problemas relacionados con hardware, software y redes, garantizando un flujo de trabajo eficiente y controlado.

Usuario final

Es la persona que detecta un incidente dentro del Data Center o en los servicios asociados. Sus funciones son:

- Reportar incidentes
- Describir el problema
- Consultar el estado del ticket

Técnico de soporte

Es el encargado de resolver los incidentes reportados. Sus funciones incluyen:

- Recibir notificaciones de incidentes
- Analizar y diagnosticar problemas
- Cambiar el estado del ticket (en proceso, resuelto)
- Registrar soluciones aplicadas

Administrador del sistema

Es el responsable de la gestión general del sistema. Sus funciones son:

- Supervisar todos los incidentes
- Asignar técnicos a los tickets
- Gestionar usuarios y roles
- Generar reportes



REQUERIMIENTOS FUNCIONALES

El sistema debe permitir:

- Crear tickets de incidentes
- Clasificar incidentes (Hardware, Software, Red)
- Asignar técnicos según especialidad
- Cambiar estados del ticket:
 - Pendiente
 - En proceso
 - Resuelto
- Notificar automáticamente a los técnicos
- Consultar historial de incidentes
- Registrar soluciones aplicadas

REQUERIMIENTOS FUNCIONALES

El sistema debe cumplir con:

- Escalabilidad: Capacidad de adaptarse a un aumento de incidentes o usuarios.
- Mantenibilidad: Código estructurado y fácil de modificar.
- Disponibilidad: Acceso continuo al sistema.
- Seguridad: Control de acceso según roles.
- Trazabilidad: Registro de cambios y seguimiento de versiones mediante Gitflow.
- Rendimiento: Respuesta rápida en la gestión de tickets.

ANÁLISIS TÉCNICO

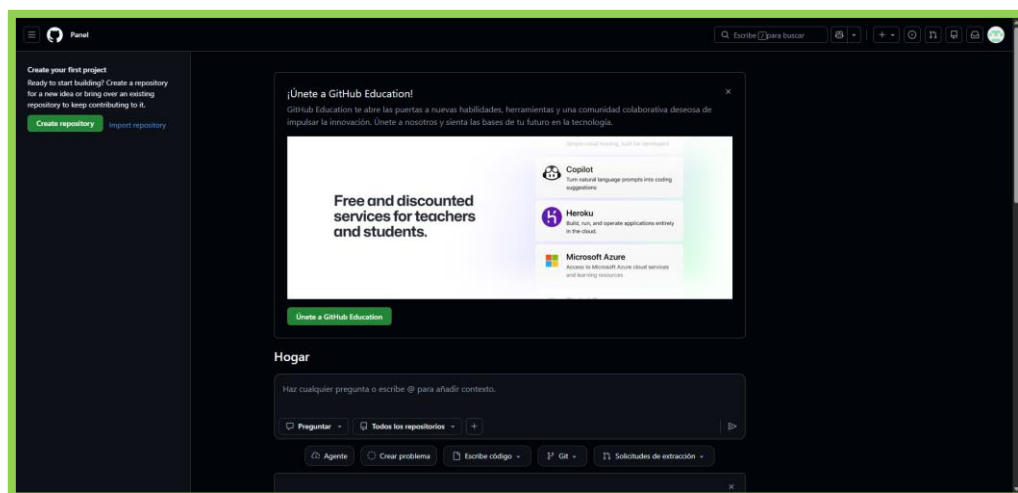
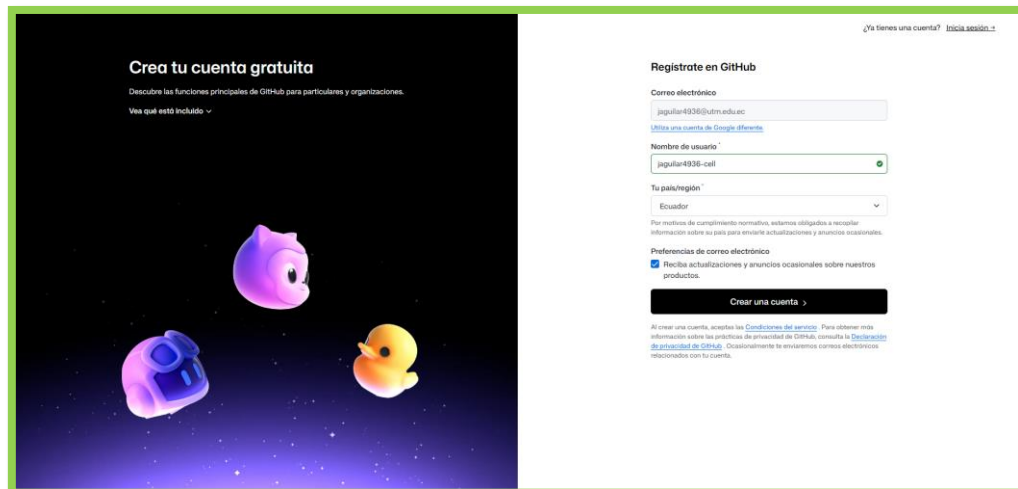
El sistema requiere una arquitectura que permita separar responsabilidades, facilitar el mantenimiento y garantizar la reutilización del código. Además, se debe integrar mecanismos de notificación automática y control centralizado de instancias. Se debe de considerar el uso de patrones de diseño como Singleton, Observer y Factory Method, así como una arquitectura basada en MVC para organizar adecuadamente la lógica del sistema.



- **INSTALACION DE GITHUB**

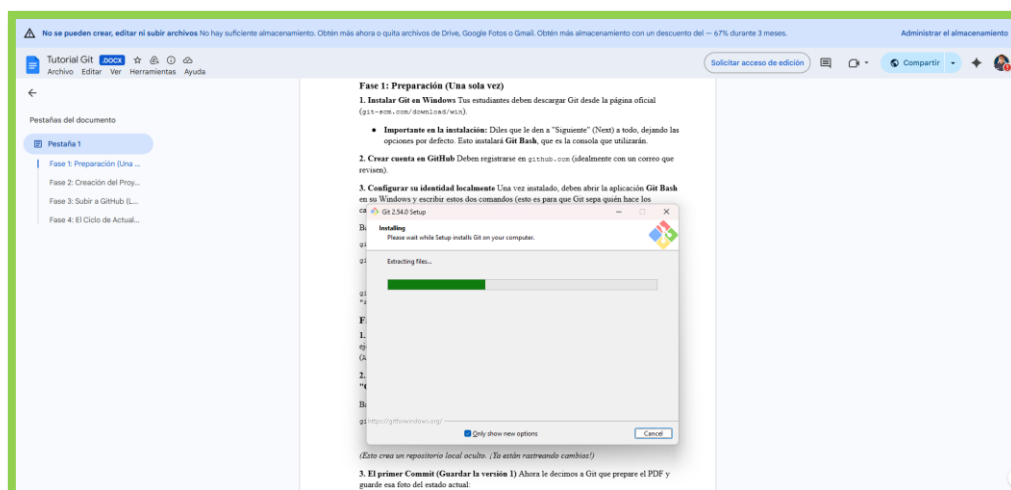
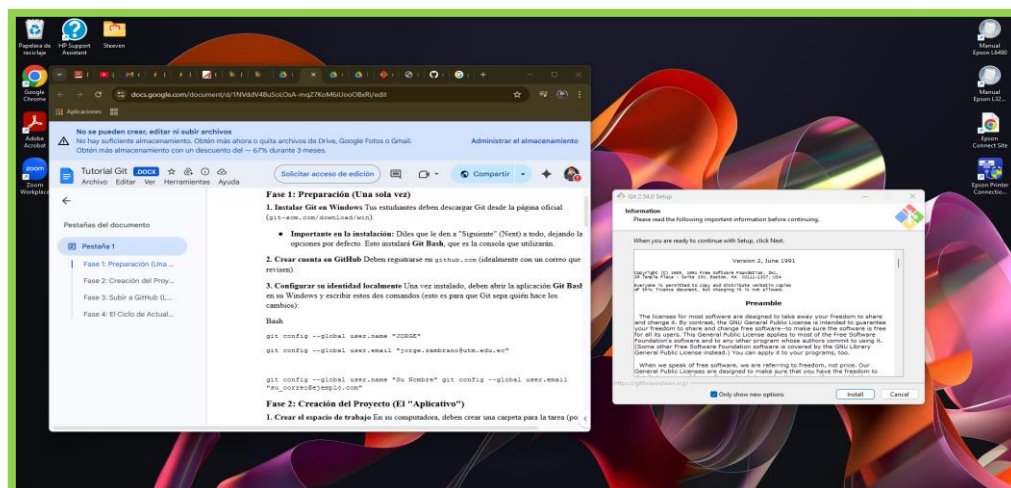
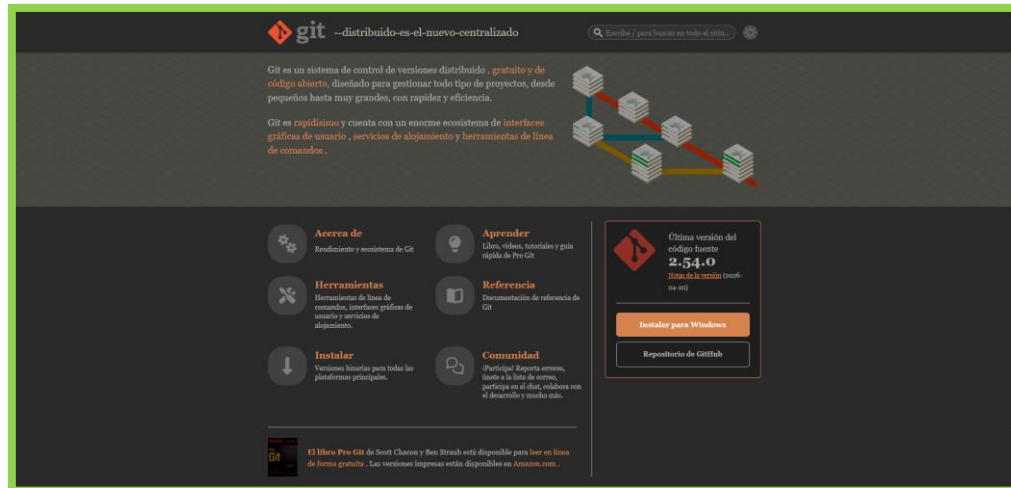


- **Creación de cuenta en GitHub**



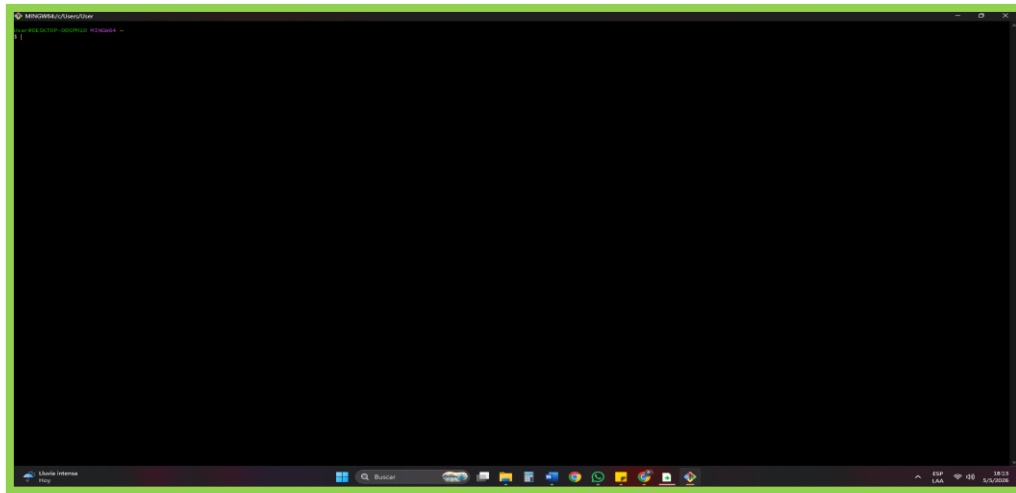


• Instalación de Git

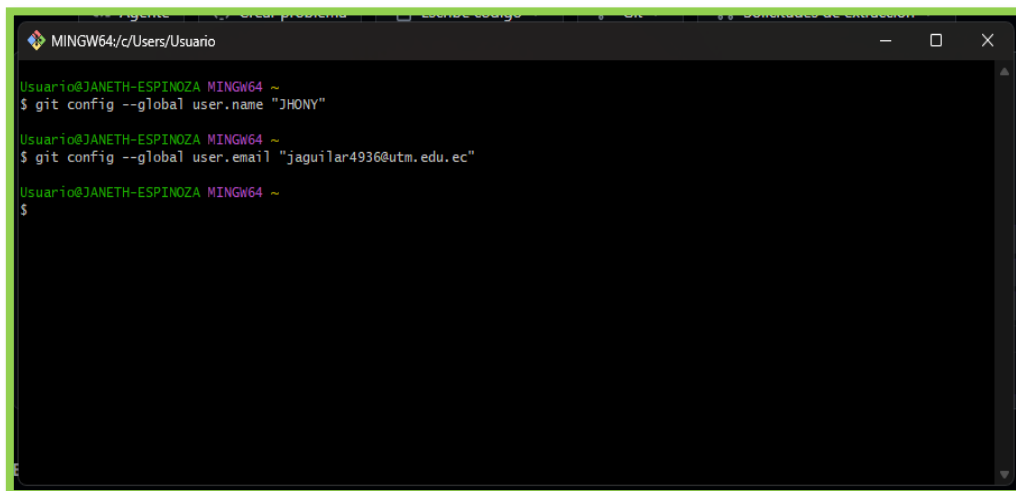




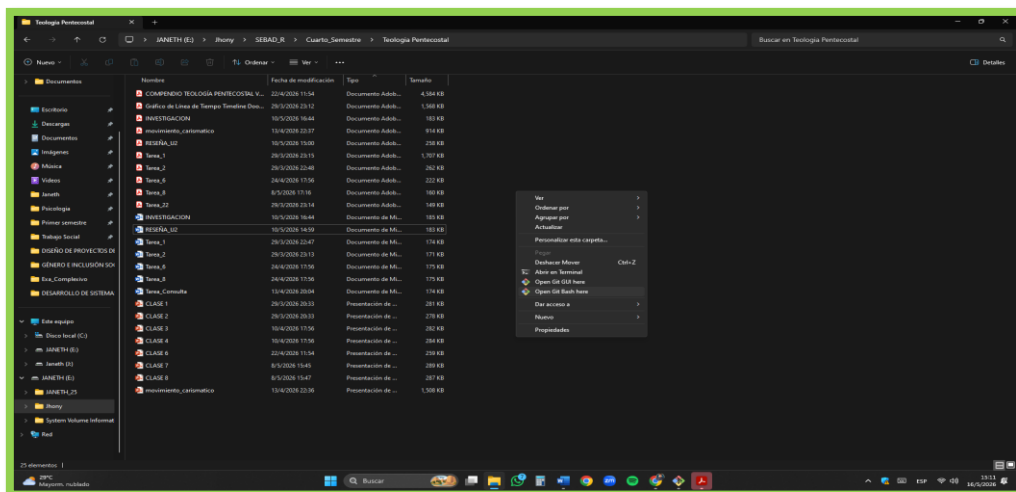
- Creación de MING

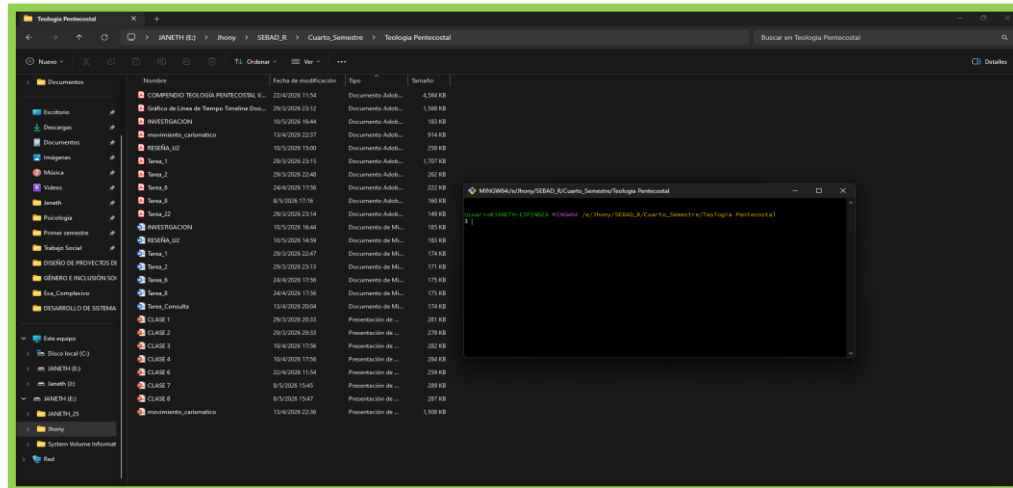


- Creación de identificadores internos



- Creación del Repositorio

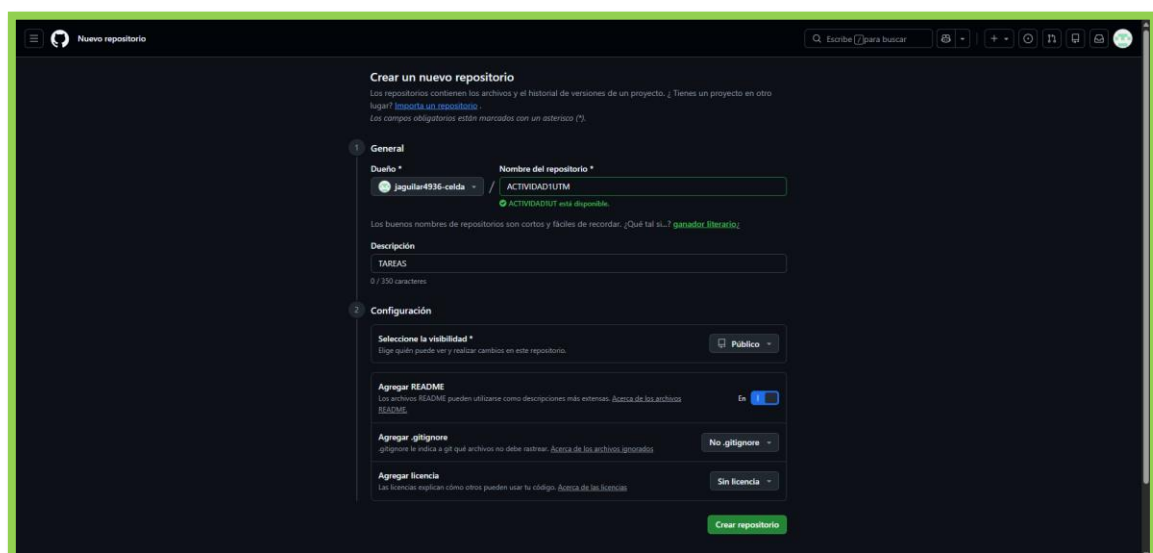


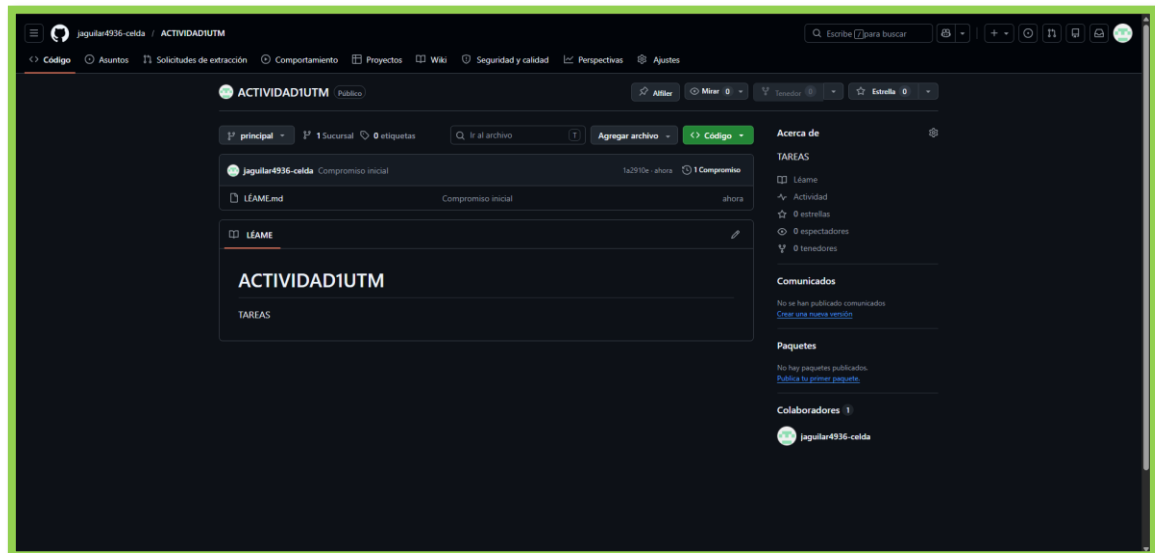


- **Archivo guardado en el repositorio en la rama master**

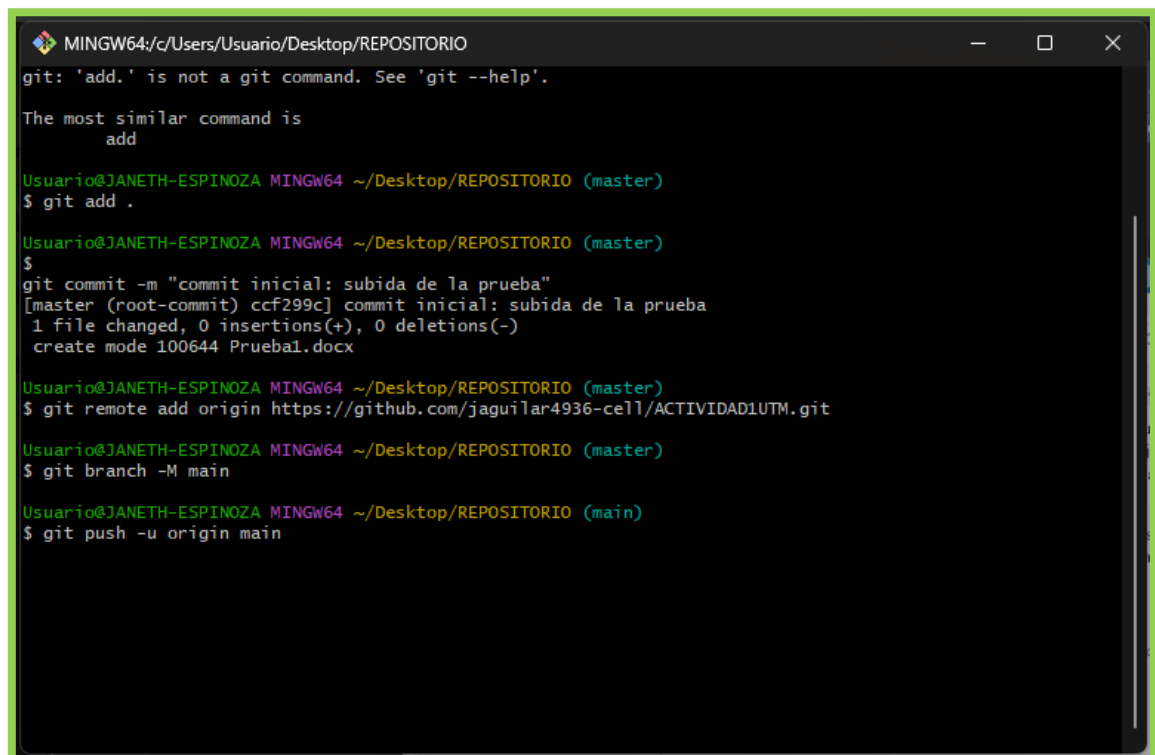


- **Creación del repositorio**



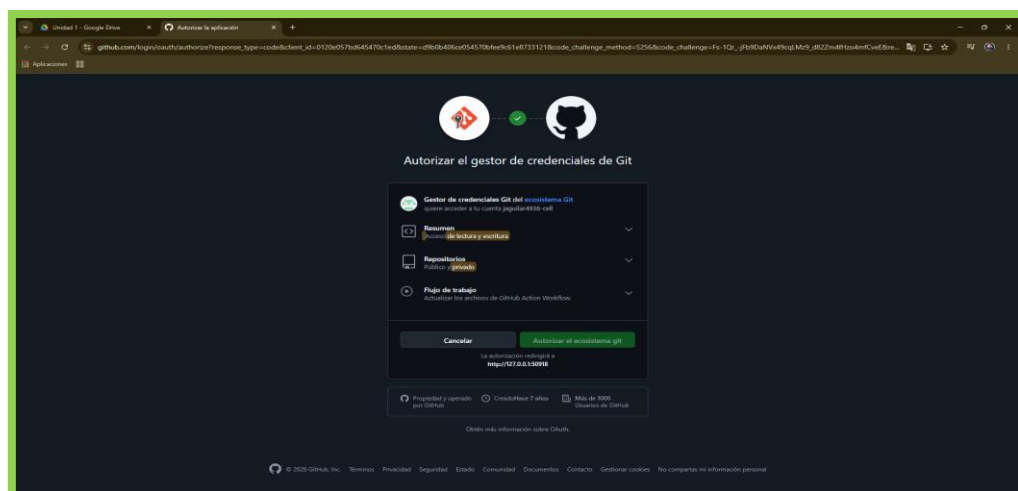
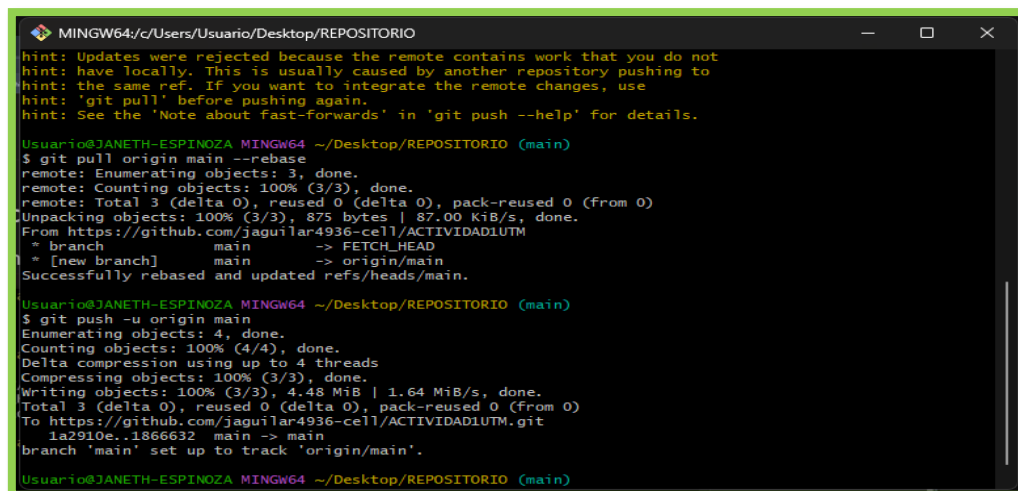
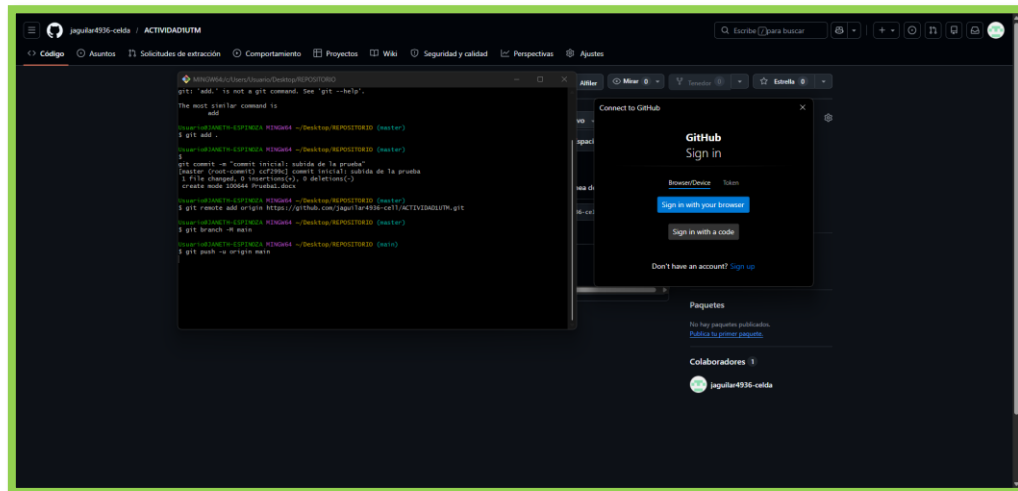


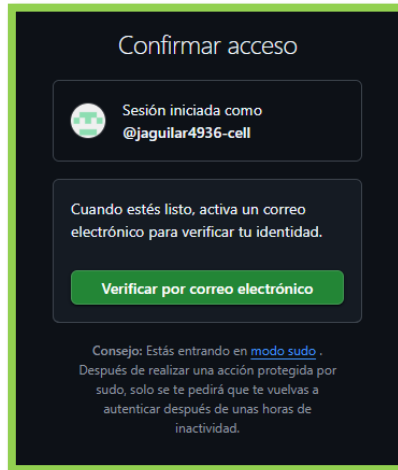
- Repositorio publico Vacío
- Creación de Branch
- Envío al Main



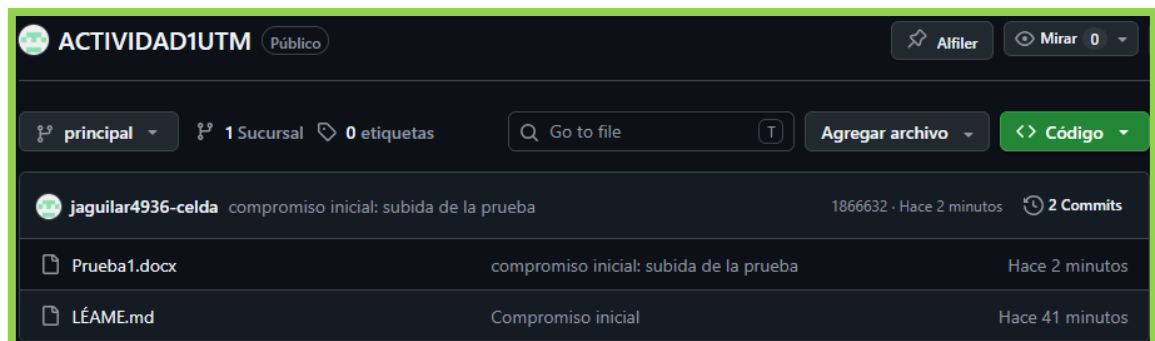
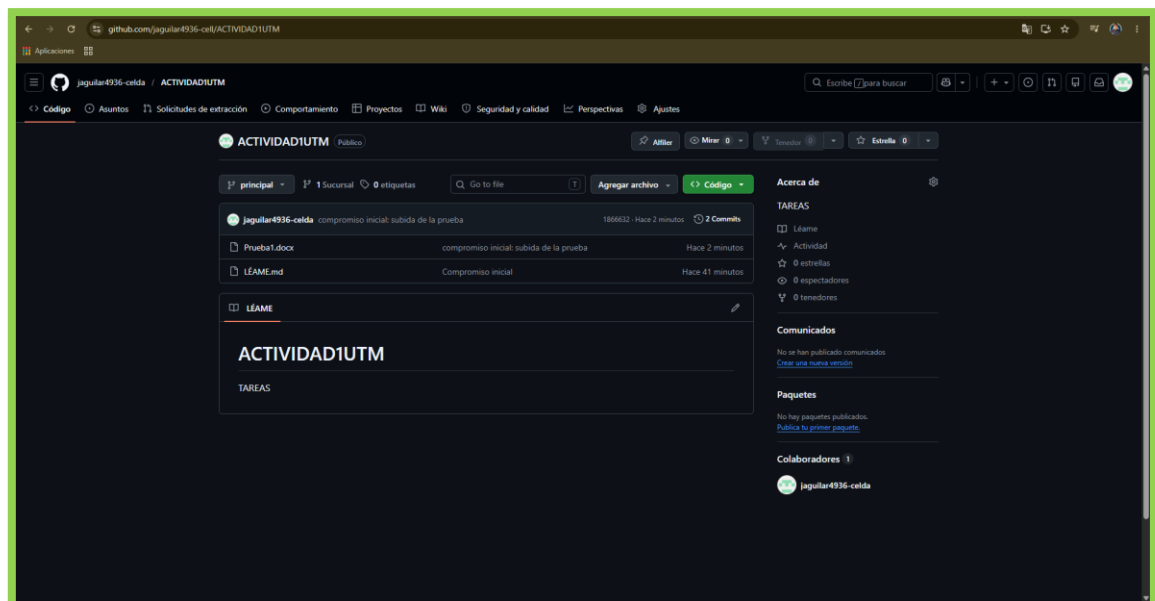


- Conexión del Git





- Creada exitosamente la rama





DISEÑO DEL SISTEMA

- 1) Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).**

Estructura de ramas:

- master (o main): contiene la versión estable del sistema
- develop: integra los avances del desarrollo
- feature/*: ramas utilizadas para desarrollar nuevas funcionalidades

Ejemplo aplicado:

- feature/tickets
- feature/incidentes
- feature/asignacion-tecnico

Evidencia:

En el repositorio se deben incluir capturas que muestren:

Creación de ramas

Historial de commits

Integración de cambios (merge)

- 2) Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.**

Este diagrama permite representar la interacción entre los actores del sistema y las funcionalidades del Help Desk.

Actores:

- Usuario final
- Técnico de soporte
- Administrador



Casos de uso principales:

- Crear ticket
- Clasificar incidente
- Asignar técnico
- Cambiar estado del ticket
- Notificar incidente
- Consultar estado del ticket

Incluir graficos

3) Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID

Este diagrama representa la estructura del sistema y las relaciones entre objetos.

Clases principales:

- Usuario
 - id
 - nombre
 - rol
- Ticket
 - id
 - descripción
 - estado
 - tipo
- Incidente
 - tipo (Hardware, Software, Red)
 - prioridad
- Técnico
 - especialidad
 - disponibilidad
- GestorTickets (Singleton)
 - crearTicket()
 - asignarTecnico()
 - cambiarEstado()

Relaciones:

- Usuario → crea → Ticket
- Ticket → contiene → Incidente
- Técnico → atiende → Ticket
- GestorTickets → controla → todo el sistema

Agregar imágenes



4) Diseñar la arquitectura general del sistema integrando el flujo de versiones

El sistema se basa en el patrón Modelo-Vista-Controlador (MVC), el cual permite separar responsabilidades.

Componentes:

Modelo (Model)

- Gestiona los datos
- Ejemplo: Ticket, Usuario, Incidente

Vista (View)

- Interfaz del usuario
- Formularios para crear tickets

Controlador (Controller)

- Maneja la lógica
- Procesa solicitudes del usuario
- Conecta modelo y vista

Flujo del sistema:

1. Usuario crea un ticket (Vista)
2. El controlador procesa la solicitud
3. El modelo guarda la información
4. Se notifica al técnico
5. El técnico actualiza el estado



APLICACIÓN DE PATRONES DE DISEÑO

En el diseño del sistema Help Desk se aplican varios patrones de diseño de software con el objetivo de mejorar la organización del código, facilitar su mantenimiento y permitir la escalabilidad del sistema. A continuación, se describen los patrones utilizados y su aplicación dentro del sistema.

- **Patrón Singleton**

El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

Aplicación en el sistema:

Se utiliza en la clase GestorTickets, la cual es responsable de:

- Crear tickets
- Asignar técnicos
- Gestionar estados

Problema que resuelve:

Evita la creación de múltiples instancias que puedan generar inconsistencias en la gestión de incidentes.



- **5.2 Patrón Observer**

El patrón Observer permite que un objeto notifique automáticamente a otros objetos cuando ocurre un cambio en su estado.

Aplicación en el sistema:

Se implementa para el sistema de notificaciones:

- Cuando se crea un nuevo ticket
- Cuando cambia el estado de un incidente

Los técnicos suscritos reciben notificaciones automáticamente.

Problema que resuelve:

Evita tener que notificar manualmente a cada técnico cuando ocurre un evento.

- **Patrón Factory Method**

El patrón Factory Method permite crear objetos sin especificar la clase exacta del objeto que se va a crear.

Aplicación en el sistema:

Se utiliza para la creación de incidentes según su tipo:

- IncidenteHardware
- IncidenteSoftware
- IncidenteRed
- El sistema decide automáticamente qué tipo de incidente crear.

Problema que resuelve:

Evita condicionales complejos al momento de crear diferentes tipos de incidentes.

- **Patrón MVC (Modelo - Vista - Controlador)**

El patrón MVC separa la aplicación en tres componentes: modelo, vista y controlador.

Aplicación en el sistema:

- **Modelo:** Ticket, Usuario, Incidente
- **Vista:** interfaz de usuario (formularios)
- **Controlador:** lógica del sistema

Problema que resuelve:

Evita mezclar la lógica de negocio con la interfaz de usuario.



Documentación del diseño

- **Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.**
- **Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.**

Justificación del uso de Gitflow

Para la gestión del desarrollo del sistema, se utilizó la plataforma GitHub implementando el modelo Gitflow, el cual permite organizar el trabajo en diferentes ramas y mantener un control adecuado de versiones.

El uso de Gitflow facilita:

- El desarrollo paralelo de funcionalidades
- La integración ordenada de cambios
- La estabilidad del sistema en producción
- La trazabilidad de cada modificación realizada

Evidencia requerida:

Se incluyen capturas de pantalla del repositorio donde se evidencie:

- Creación de ramas (master, develop, feature/...)
- Commits realizados
- Proceso de merge entre ramas
- **Incluir diagramas de clases y secuencia donde se refleje la implementación de los patrones.**

Bibliografía

Diseño Estructurado de Sistemas. (2021). Retrieved 02 Mayo 2021, from [https://users.exa.unicen.edu.ar/catedras/proq1/sites/default/files/ApuntesDiagramaEstructura.Pdf](https://users.exa.unicen.edu.ar/catedras/proq1/sites/default/files/ApuntesDiagramaEstructura.pdf)

Object Oriented Design. (2021). Retrieved 02 Mayo 2021, from <https://www.professionalqa.com/object-oriented-design>

Diseño a nivel de componentes. (2021). Retrieved 04 Mayo 2021, from [https://virtual.itca.edu.sv/Mediadores/stis/37 diseo a nivel de componentes.html](https://virtual.itca.edu.sv/Mediadores/stis/37%20dise%C3%B1o%20a%20nivel%20de%20componentes.html)